Operationalizing Agent Provenance: A Structural Analysis of the Stela Pointer-First Constitution and Context Hazard Protocols

1. Executive Summary: The Governance of Stateless Intelligence

The Stela repository represents a distinct evolutionary step in the architecture of AI-assisted software development, moving away from ephemeral, prompt-based interactions toward a rigorous, provenance-backed state machine. In this architecture, the repository is not merely a passive storage medium for code; it is the active cognitive architecture of the system. The **Policy of Truth (PoT)** serves as the immutable constitution, dictating that the documentation is not a description of the system, but the law by which the system functions. This "stateless" philosophy—where "Context is not memory; it is a file"—is designed to eliminate the greatest risk in AI engineering: **Drift**.[1]

However, a critical asymmetry currently exists within Stela's operational operational fabric. While the **Skills System** (docs/library/skills/) has achieved a state of high maturity through the implementation of automated "Harvest and Promote" workflows—ensuring every skill has cryptographic provenance and is treated as a distinct operational artifact—the **Agent System** (docs/library/agents/) remains structurally immature. Agents are currently defined as static markdown files (e.g., R-AGENT-01) that often duplicate behavioral instructions found in the PoT, violating the "Pointer-first" doctrine. This static nature renders them susceptible to obsolescence and hallucination, as they lack the verifiable link to successful execution that characterizes the Skills System.

This report provides an exhaustive architectural analysis and operational specification for updating the Agent System to conform to Stela's operational specifications. By adapting the **Harvest/Promote** logic currently used for skills, Stela can transition agents from static role definitions into dynamic, provenance-backed operational assets. Furthermore, this analysis defines the mechanisms for **Context Hazard** enforcement and **Seclusion**, ensuring that agents operate in hermetically sealed environments that prevent context contamination and align with the Filing Doctrine.[1]

2. Constitutional Foundations: The Policy of Truth

To operationalize the requested updates to the Agent System, one must first dissect the

constitutional framework that governs Stela. The **Policy of Truth (PoT.md)** is the singular source of authority, establishing the "Physical Laws" of the repository. Understanding these laws is prerequisite to modifying the agent architecture, as any deviation constitutes a system failure.

2.1 The Physics of Stela

The PoT establishes a governance model based on "statelessness." In traditional software environments, "context" is often a vague accumulation of tribal knowledge, Slack history, and implicit assumptions. In Stela, the PoT asserts that **"Context is not memory; it is a file."** If an instruction, rule, or fact is not present in the loaded context files (the Manifest), it does not exist for the purpose of the current session.[1] This axiom is the foundation of **Anti-Drift Governance**, ensuring that the system state is always deterministic and reproducible.

2.1.1 The Filing Doctrine

The Filing Doctrine defined in PoT Section 1.1 separates the repository into four distinct jurisdictions, each governed by specific physics and intent. This separation is crucial for the proposed "Seclusion" architecture.

Jurisdiction	Path	Function	Physics
Run	ops/	Executable Kernel	Contains binaries, manifests, and scripts. This is the "enforcement arm" of the constitution. Logic here must be deterministic.
Explain	docs/	Legislative Branch	Contains manuals, rationale, and the PoT itself. It explains *why* the system functions. Agents and Skills live here as "Library" artifacts.
Work	projects/	Payload Sector	Where value is created. Projects are consumers of

			platform governance but must never define governance that contradicts the PoT.
Trash	storage/	Ephemeral Zone	Local artifacts, dumps, and handoff files. These are never canon and are excluded from the repository's permanent record via .gitignore.

The "Context Hazard" referenced in the user query stems from a violation of this doctrine—specifically, when artifacts from the "Explain" layer (like the entire library of skills) are forced into the "Run" layer's active context without filtration, overwhelming the AI's cognitive window.[1]

2.1.2 Staffing Protocol and Jurisdiction

The Staffing Protocol (PoT Section 4.1) defines a rigid hierarchy of authority that must be encoded into the new Agent definitions.[1]

- **Operator (Human):** Holds absolute sovereignty. Only the Operator can commit, push, or merge. The Operator owns "secrets" and final approvals. The Operator is the biological validation layer.
- **Integrator (Lead AI):** The guardian of the PoT. The Integrator's role is not to write code, but to maintain "structural integrity" and detect "Drift." The Integrator generates **Dispatch Packets (DPs)**—immutable contracts for work. The Integrator is the *State Manager*.
- **Contractor (Guest AI):** The executor. Contractors are instantiated to perform specific tasks defined in a DP. They act as "Guest AIs" and must adhere to strict behavioral logic (no contractions, absolute literalism). The Agent definitions (R-AGENT-XX) are essentially **Role Cards** for these Contractors.

2.2 The Pointer-First Doctrine

The user query explicitly mandates a "Pointer-first agent constitution" with "no duplicated

canon text".[1] This aligns with Stela's llms.txt philosophy found in the dump.[1]

In many AI repositories, system prompts are monolithic blocks of text that repeat the same instructions (e.g., "You are a helpful assistant," "Do not use markdown in titles") across multiple files. Stela rejects this redundancy.

- **SSOT (Single Source of Truth):** Information must exist in exactly one place. All other references must be links or pointers to that canonical location.
- **Efficiency:** Repeating the behavioral logic standard (e.g., "No contractions") in every Agent file (R-AGENT-01 through 06) creates a maintenance nightmare. If the PoT changes to allow contractions, every agent file is instantly "Drifted" (out of date).
- **Implementation:** The new Agent files must strip all behavioral instructions and simply *point* to PoT.md Section 2.

3. The Skills System Precedent: A Model for Provenance

The **Skills System** (docs/library/skills/) represents the "target state" for the Agent System. It has successfully operationalized the concept of "Harvest and Promote," solving the problem of how to capture ephemeral AI workflows into durable assets. By dissecting the ops/lib/scripts/skill.sh script provided in the dump, we can reverse-engineer the logic required for the new Agent Protocol.[1]

3.1 Forensic Analysis of the Harvest Engine

The skill.sh script is the mechanical heart of the Skills System. It does not relying on human memory to write documentation; it relies on **forensic capture**.

3.1.1 Provenance Capture Mechanisms

When a Contractor successfully completes a task, the Operator invokes ops/lib/scripts/skill.sh harvest. This command triggers a provenance capture sequence defined in ops/lib/scripts/heuristics.sh.[1]

- **DP-ID Capture:** The script extracts the active Dispatch Packet ID from TASK.md, linking the skill to a specific authorized work order.
- **Git State Forensics:** It captures the HEAD commit hash and the branch name. This provides a cryptographic link to the exact state of the codebase when the skill was validated.
- **Friction Context Analysis:**
  - **Hot Zone Detection:** The script uses git diff --numstat to identify which files changed the most during the session. If the Contractor spent 90% of its effort in ops/bin/dump, the script automatically tags the "Hot Zone" as ops/bin/dump.
  - **High Churn Detection:** It analyzes git log to find files that were touched frequently, indicating instability or complexity ("High Churn").

- **Immutable Block:** These data points are written into a YAML-like "Provenance" header in the draft markdown file. This block is immutable; it proves that the skill was not "hallucinated" by an LLM in a vacuum but was born from actual, verified work.

3.1.2 The Semantic Collision Check

The script includes a check_semantic_collision function.[1] It scans existing skills in docs/library/skills/ and current drafts in storage/handoff/ for keyword overlaps. This prevents the "Library Bloat" drift, where multiple slightly different versions of "How to Refactor" clog the context. This logic must be ported to the Agent System to prevent duplicate personas (e.g., "Code Cleaner" vs. "Refactor Specialist").

3.2 The Promotion Gate: Registry Mechanics

Once harvested, a skill exists as a "draft" in storage/handoff/. It is not yet canon. It must pass the **Promotion Gate**, managed by the promote function in skill.sh.[1]

- **Review:** The Operator reviews the draft for hallucinations or placeholders (e.g., TODO, ENTER_SOLUTION). The script explicitly greps for these tokens and fails if they are present.
- **Promotion:** The command ops/lib/scripts/skill.sh promote <draft_path> moves the draft to docs/library/skills/S-LEARN-XX.md.
- **Registry Lock:** The script calculates the next available ID (e.g., S-LEARN-08) and registers the skill in docs/library/INDEX.md. This ensures the **Library Index** remains the authoritative map of the system's capabilities.

3.3 Context Hazard Guardrails

A critical innovation in the Skills System is the **Context Hazard** protocol.[1]

- **The Hazard:** Including the entire library of skills (docs/library/skills/) in the global context (ops/lib/manifests/CONTEXT.md) is defined as a "failure state" in PoT Section 1.2.[1]
- **The Reason:** Loading 50 different "how-to" guides consumes the AI's limited context window and creates "solution bias," where the AI tries to force-fit irrelevant patterns (e.g., applying a Python refactoring skill to a Bash script).
- **The Guardrail:** The tools/lint/context.sh script enforces this. It scans the CONTEXT.md manifest. If it finds any path pointing to docs/library/skills (or agents or tasks), it triggers a fatal error, blocking the open process.
- **JIT Loading (Precursor to Seclusion):** Skills are loaded **Just-In-Time (JIT)**. They are only included in a session if the Dispatch Packet (DP) explicitly requests them by ID or path.

4. Current Agent Architecture: The Drift Risk

The current Agent System (docs/library/agents/) operates on an older, static model that

predates the Skills System's rigor. While it defines roles like **Architect** (R-AGENT-01) and **Code Reviewer** (R-AGENT-02), these definitions lack provenance and violate the Pointer-First doctrine.[1]

4.1 Structural Analysis of Existing Agents

Examination of the R-AGENT files in the dump reveals significant structural weaknesses:

- **R-AGENT-01 (Architect):** Contains broad, static lists of principles ("Modularity," "Scalability") and process steps ("Requirements Gathering"). These are textbook definitions, not operational protocols specific to Stela. It essentially "lectures" the AI on what an architect is, rather than defining *this* specific architect's boundaries within the Stela repo.
- **R-AGENT-02 (Code Reviewer):** Explicitly lists "Security Checks" and "Code Quality" rules. While valuable, these rules are hard-coded in the agent file. If the project updates its security standards (e.g., switching from JWT to PASETO), this agent file becomes stale ("Drift").
- **Duplicated Governance:** Crucially, these files implicitly or explicitly repeat behavioral rules. R-AGENT-02 instructs "Provide feedback organized by priority," which overlaps with the PoT's "Quantitative Reporting" standard.

The Failure Mode: Because these files are static and hand-written, they drift from the actual capabilities of the AI models and the evolving constraints of the repository. There is no proof that R-AGENT-01 has ever successfully architected anything in Stela; it is a theoretical construct.

4.2 The "Context Hazard" of Agents

Currently, the PoT explicitly lists docs/library/agents as a **Context Hazard**.[1]

- **Negative Constraint:** "ops/lib/manifests/CONTEXT.md must exclude docs/library/agents".[1]
- **Current Friction:** Operators likely struggle with this because to use an agent, they must currently copy-paste the agent definition or manually load it, risking context contamination.
- **Solution Requirement:** The new system must allow for **JIT Injection** of agents without violating the manifest rules. The Agent file must be "loadable" on demand via the Dispatch Packet.

4.3 The "Pointer-First" Deficiency

The current agents are "Content-First." They contain the knowledge.

The new requirement is "Pointer-First." The Agent should contain **Connections**.

- Instead of listing "How to check security," the Agent should point to S-LEARN-05 (Security Protocols).

- Instead of listing "How to dump state," the Agent should point to ops/bin/dump.
- Instead of listing "How to speak," the Agent should point to PoT.md.

This transforms the Agent from a "Textbook" into a "Router."

5. Operational Specification: The Agent Provenance Protocol

To align the Agent System with the Stela Operational Specs, we define a new lifecycle: **Agent Harvesting and Promotion**. This involves treating Agent definitions not as static text, but as **captured cognitive snapshots** of successful operational states.

5.1 The New Agent Lifecycle (Harvest -> Promote)

We propose extending ops/lib/scripts/ with a new logic—either by expanding skill.sh or creating a dedicated agent.sh. Given the distinct metadata, a dedicated ops/lib/scripts/agent.sh is recommended.

5.1.1 Logic Specification: harvest-agent

When a Contractor (Guest AI) performs exceptionally well on a specific task (e.g., "Refactoring the ops/bin/dump script"), the Operator must "Harvest" that specific persona.

- **Command:** ./ops/lib/scripts/agent.sh harvest --name "Dump Refactor Specialist" --dp "DP-OPS-0029"
- **Provenance Capture:** The script must capture:
  - **The System Prompt Stack:** The exact combination of Open Prompt + DP + loaded Context files used.
  - **The Output Pattern:** The specific way the AI structured its response (e.g., the exact diff format, the receipt style).
  - **The Friction:** The constraints that forced the AI into this optimal state (Hot Zone, High Churn).
- **Artifact:** Creates storage/handoff/agent-draft-YYYYMMDD-slug.md.

5.1.2 Logic Specification: promote-agent

The promotion logic must enforce the **Pointer-First** doctrine via automated linting *before* allowing the promotion.

- **Command:** ./ops/lib/scripts/agent.sh promote <draft_path>
- **Pre-Flight Checks (Linter):**
  - **PoT Duplication Check:** The script scans the draft for phrases like "No contractions," "Do not edit main," or "Policy of Truth." If found, it **fails**. The error message instructs the Operator to remove the text and replace it with a pointer to PoT.md.
  - **Context Hazard Check:** Ensures the agent does not attempt to hard-code lists of

skills. It must use pointers.

- **Registry Action:**
  1. Assigns the next sequential ID: R-AGENT-XX.
  2. Moves file to docs/library/agents/R-AGENT-XX.md.
  3. Updates docs/library/INDEX.md with the new agent.
  4. Updates SoP.md (State of Play) to record the new capability.[1]

5.2 Revised Agent Structure (R-AGENT-XX Template)

The new Agent definition file must follow this strict schema to ensure compatibility with **Seclusion** and **Context Hazard** rules. This schema completely replaces the content-heavy format of the current R-AGENT-01.

Table 1: The Pointer-First Agent Schema

Section	Content Strategy	Pointer Target (SSOT)	Rationale
Header	Name, Description, Model	N/A	Defines the persona identity.
Provenance	Immutable capture data	ops/lib/scripts/agent.sh	Proof of Work. Prevents hallucinated capabilities.
Constitution	"Follow these laws."	PoT.md	Removes behavioral duplication (e.g., linguistic precision).
Jurisdiction	"Where you can work."	docs/GOVERNANCE.md	Defines "Tone Lanes" and "Non-Negotiables."
Output	"How to report."	TASK.md	Enforces the "Receipt" and "Verification" formats.

Toolchain	"What you can run."	ops/bin/, tools/lint/	Explicitly authorizes binary usage (e.g., ops/bin/dump).
Skills	"What you know."	docs/library/skills/S-LEARN-XX	JIT-loads specific technical know-how (e.g., Security, React).
Scope	"Seclusion boundary."	projects/ (Allowlist)	Defines the "Seclusion" boundary for the agent.

5.3 Context Hazard Enforcement

The existing "Context Hazard" logic in PoT.md and ops/bin/llms must be rigorously maintained.

- **Negative Constraint:** ops/lib/manifests/CONTEXT.md **MUST NOT** contain docs/library/agents/R-AGENT-XX.md.
- **JIT Injection:** Agents are injected only via the **Open Prompt** or the **Dispatch Packet**.
  - **Scenario:** The Operator runs ./ops/bin/open --intent="Refactor". The Operator manually selects the agent: "Act as R-AGENT-05 (Refactor Cleaner)."
  - **Mechanism:** The open binary (or the prompt) reads docs/library/agents/R-AGENT-05.md and injects its content *dynamically* into the session context, without permanently adding it to the llms.txt or global manifest. This ensures the "hazard" is contained to the specific session.

6. Seclusion Doctrine: The Future of Project Development

The user query mentions "SECLUSION TO BE EVENTUALLY USED ALONG WITH SKILLS AND TASKS".[1] This points to the next evolution of Stela's architecture: **Project Seclusion**.

6.1 The Problem with Global Context

Currently, the "Small Bundle" (llms-small.txt) and "Full Bundle" (llms-full.txt) are global.[1] They load the platform's operating system (ops/, docs/). However, as projects/ grow (e.g., projects/php-nuke-ce or new payloads), loading the global platform context alongside project-specific code risks two failures:

1. **Context Overflow:** The AI forgets platform rules because the project code is too large.

2. **Scope Leak:** The AI tries to apply platform governance rules (e.g., "Don't edit main") to a project that might use a different workflow (though Stela forbids this, the risk of confusion exists).

6.2 Defining Seclusion

Seclusion is the architectural state where an Agent operates within a **hermetically sealed context** containing *only* the artifacts necessary for the specific Dispatch Packet.

The Seclusion Equation:

$$SeclusionContext = PoT + DP + R_AGENT + JITSkills + TargetProjectFiles$$

Crucially, **Seclusion** excludes:

- Unrelated docs/ files.
- Unrelated ops/ scripts (unless in the toolchain).
- Other projects/.
- The rest of the library/.

6.3 Implementing Seclusion via ops/bin/project

The ops/bin/project binary [1] is the vehicle for Seclusion.

- **Current State:** It assembles OPEN + dump (platform) + STELA.md (project).
- **Target State:** It should assemble a **Secluded Bundle**:
  - **Base:** PoT.md, TASK.md, SoP.md.
  - **Agent:** The specific R-AGENT-XX requested.
  - **Project:** The specific projects/<id> files (via ops/bin/dump --scope=project).
  - **Skills:** Only the S-LEARN items cited in projects/<id>/STELA.md or the DP.

This prevents "Context Hazard" by default. The Agent literally *cannot* hallucinate about other parts of the system because they do not exist in its universe.

7. Deep Dive: Forensic Analysis of Repository State

To ensure this report is grounded in the "Policy of Truth," we must verify the current state of the repository artifacts provided in the dump [1] to validate the feasibility of these proposals.

7.1 PoT.md (The Constitution)

- **Section 1.2 (Axioms):** Explicitly defines "Context Hazard" regarding docs/library/agents.[1] This confirms that simply "adding agents to the context" is illegal. They *must* be JIT loaded.
- **Section 4.1 (Staffing):** Defines the "Contractor." The Agent definitions (R-AGENT) are

essentially **Role Cards** for these Contractors.

- **Section 4.2 (Behavioral Logic):** Mandates "Reuse-first." This supports the argument for "Harvesting" agents. We should not invent a new "Code Reviewer" every time; we should reuse the proven R-AGENT-02.

7.2 ops/bin/dump (The Observer)

- **Logic:** Uses git ls-files for deterministic selection.[1]
- **Implication:** Any new Agent file created by harvest will be immediately visible to the system *only if* it is committed. However, dump supports specific scopes (platform, project).
- **Seclusion Alignment:** ops/bin/dump already supports --scope=project. This mechanism is ready-made for Seclusion. A "Secluded Agent" would consume a dump --scope=project rather than a dump --scope=full.

7.3 ops/lib/scripts/skill.sh (The Mechanism)

- **Provenance Block:** The script generates a ## Provenance header with "Captured," "DP-ID," "Branch," "HEAD," "Friction Context".[1]
- **Application to Agents:** This exact block should be copied to the new Agent generation logic. Knowing *when* an Agent persona was defined (and under what friction conditions) allows the Operator to trust the agent. An agent harvested during a "High Churn" critical incident is battle-tested; an agent written manually is theoretical.

8. Strategic Recommendation: The "Agent Constitution" Update

Based on this analysis, the following specific updates are proposed to docs/library/agents/ and the associated Ops Kernel.

8.1 Rename and Refactor AGENTS.md (The "Pointer Map")

The user query mentions AGENTS.md.[1] Currently, docs/library/agents/ contains individual files. We should create or update docs/library/agents/INDEX.md (or AGENTS.md if preferred, though INDEX aligns with the library pattern) to serve as the **Constitutional Pointer Map**.

Proposed Content for docs/library/agents/INDEX.md:

Agent Registry (Pointer-First)

Philosophy

Agents are JIT-loaded personas. They do not exist in the global context.

They are harvested from successful DPs and promoted to this registry.

Registry

ID	Name	Provenance (DP)	Specialization
R-AGENT-01	Architect	DP-OPS-0010	System Design, Trade-offs
R-AGENT-02	Code Reviewer	DP-OPS-0012	Security, Style, Perf
R-AGENT-03	Doc Updater	DP-OPS-0015	AST Analysis, Codemaps

8.2 Update R-AGENT-XX Files to "Pointer-First" Format

We must rewrite the existing R-AGENT files (01-06) to strip duplicated governance text.

Example Rewrite: R-AGENT-02 (Code Reviewer)

- **Remove:** "Linguistic Precision: Avoid contractions." (This is in PoT).
- **Remove:** "No Direct Pushes." (This is in PoT).
- **Add:** "## 1. Governance Pointers" -> PoT.md.
- **Focus:** Expand the "Security Checks" and "Code Quality" sections using the **Zenith** pattern.[1] The agent should inherently know *how* to check for Stela-specific patterns (e.g., "Check for docs/library inclusion in CONTEXT.md").

8.3 Implement ops/lib/scripts/agent.sh

Create the automation to enforce this lifecycle.

- **Function harvest:** Scrapes the current chat session (if accessible) or prompts the user to paste the "winning" persona instructions. Wraps them in the Provenance block.
- **Function promote:** Validates that the draft does *not* contain duplicated PoT text (linting for "No contractions" rules inside the agent file itself—redundancy is drift). Moves to docs/library/agents/ and updates INDEX.md.

9. Context Hygiene and the "Seclusion" Roadmap

The transition to a fully "Secluded" Agent system is the final step in hardening the Stela

repository.

9.1 The "Open Porcelain" Concept

The ops/bin/open script currently detects "Porcelain" (uncommitted changes) and saves them.[1] In a Secluded workflow, "Open Porcelain" becomes the **Working Memory** of the Agent.

- **Workflow:**
  1. Agent starts in Seclusion (PoT + DP + Project Files).
  2. Agent makes changes (Porcelain).
  3. Agent *cannot* see the rest of the repo.
  4. Agent requests verification. The **Integrator** (running in Global Context, not Secluded) runs the gates (repo-gates).
  5. Feedback is passed back to the Secluded Agent.

This strictly enforces the **Staffing Protocol** where the Contractor (Secluded) executes logic, but the Integrator (Global) maintains integrity.

9.2 Immediate Action Items (Next Steps)

To execute this research proposal, the Operator should:

1. **Draft DP-OPS-XXXX (Agent System Refactor):**
   - **Objective:** Refactor docs/library/agents/ to Pointer-First standard.
   - **Task:** Create ops/lib/scripts/agent.sh (cloned/adapted from skill.sh).
   - **Task:** Rewrite R-AGENT-01 through 06 to strip PoT duplications.
2. **Verify Context Hazard:** Run tools/lint/context.sh to ensure no agent files have leaked into the manifest.
3. **Pilot Harvest:** Run a complex refactor task, then "Harvest" the resulting persona to create R-AGENT-07 (e.g., "Migration Specialist").

10. Conclusion

The Stela repository's strength lies in its refusal to rely on implicit knowledge. By applying the **Harvest and Promote** pattern—proven in the Skills System—to the Agent System, Stela closes a critical governance gap. Agents will cease to be static, potentially drifting definitions and will become **provenance-backed operational artifacts**.

This shift enables the eventual implementation of **Seclusion**, where agents operate in high-hygiene, context-safe environments, interacting with projects/ without the risk of platform contamination. This is the fulfillment of the **Pointer-First Constitution**: a system where every instruction can be traced to a single source of truth, and every active agent is a verified product of the system's own history.

Detailed Analysis: Stela Operational Specs & Agent Protocol

1. Introduction: The Stela "State Machine"

Stela is not merely a codebase; it is a text-based state machine where the file system constitutes the memory, logic, and law of the operation. The **Policy of Truth (PoT)** serves as the immutable kernel, defining the physics of this universe.[1] In this environment, "Agents" are not vague AI personalities; they are distinct, file-backed operational states that must be managed with the same rigor as compiled binary code.

The user's request focuses on a critical evolution of this system: upgrading the **Agent System** (R-AGENT-XX) to match the maturity of the **Skills System** (S-LEARN-XX). This involves adopting "Pointer-first" architecture, strict "Context Hazard" guardrails, and preparing for "Seclusion."

This report dissects the existing machinery of Stela—specifically the ops/ kernel and docs/library/ registries—to engineer a compliant, robust Agent Provenance Protocol.

2. The Ops Kernel: Machinery of Truth

To propose a valid update to the Agent System, we must first understand the machinery that will enforce it. The ops/ directory is the "Run" jurisdiction.[1]

2.1 The "Open/Dump" Loop

The fundamental heartbeat of Stela is the **Open/Dump Loop**.

- **Open (ops/bin/open):** This tool initializes a session. It captures the "Freshness Gate" (Git Commit Hash, Branch Name) and injects the "Intent" and "Dispatch Packet (DP)" into the context.[1] It establishes the *temporal reality* of the agent.
- **Dump (ops/bin/dump):** This tool serializes the repository state into a single text stream for the AI. Crucially, it supports **Scopes** (platform, full, project). This scoping capability is the technical foundation for **Seclusion**—allowing an agent to see *only* what is relevant.[1]

2.2 The Context Bundler (ops/bin/llms)

The ops/bin/llms binary is the gatekeeper of what the AI "knows".[1]

- **Manifest Supremacy:** It reads ops/lib/manifests/CONTEXT.md to decide what files to include.

- **Hazard Exclusion:** It explicitly rejects docs/library/agents, docs/library/tasks, and docs/library/skills. This is the code-level enforcement of the **Context Hazard** axiom.
- **Profiles:** It supports profiles like --profile=architect or --profile=security. This is a primitive form of Seclusion, injecting specific knowledge layers on top of the base context.[1]

Implication for Agents: The new Agent System must integrate with ops/bin/llms. Instead of "Profiles" being hard-coded lists, they should likely be driven by the R-AGENT definition itself.

3. The Skills System: The Gold Standard

The **Skills System** is the benchmark for the proposed Agent update. It has successfully solved the problem of "Hallucinated Capabilities" by enforcing **Provenance**.

3.1 Anatomy of a Skill (S-LEARN)

Reviewing the S-LEARN files (e.g., S-LEARN-01, S-LEARN-07) reveals a rigid schema [1]:

1. **Provenance Block:** Captured: YYYY-MM-DD, Origin: DP-OPS-XXXX, Friction Context: Hot Zone. This proves the skill is real.
2. **Scope:** "Production payload work only."
3. **Invocation Guidance:** "Use this skill when..."
4. **Drift Preventers:** "Stop conditions," "Anti-hallucination."
5. **Procedure:** The executable steps.

3.2 The Harvest/Promote Workflow

The workflow is circular and self-reinforcing [1]:

1. **Work:** A Contractor executes a DP.
2. **Harvest:** The Operator runs ops/lib/scripts/skill.sh harvest. The script scrapes the session/git-state and creates a draft.
3. **Refine:** The draft is reviewed in storage/handoff/.
4. **Promote:** The Operator runs promote. The skill becomes Canon (docs/library/skills/S-LEARN-XX.md).
5. **Register:** The index (docs/library/INDEX.md) is updated.

Insight: This workflow turns "work" into "asset." The user wants to apply this to **Agents**. An Agent shouldn't just be "written"; it should be "harvested" from a session where an AI successfully embodied a specific persona (e.g., "The ruthless code reviewer").

4. Agent System Analysis: The "Static" Flaw

Currently, Agents (docs/library/agents/) are static.

- **Drift Risk:** R-AGENT-01 (Architect) [1] defines how an architect *should* behave. But if the

actual LLM models evolve, or if the repository complexity grows, this static file might become obsolete.
- **Redundancy:** Agents currently contain instructions like "No Direct Pushes" or "Linguistic Precision." These are global rules defined in PoT.md. Including them in R-AGENT files violates **Pointer-First** design (Don't Repeat Yourself). If PoT.md changes, every Agent file is now out of date.

4.1 The Pointer-First Imperative

The user explicitly asks for a "Pointer-first agent constitution."

- **Current State:** Content-heavy agent files.
- **Target State:** Context-light, Pointer-heavy agent files.
  - Instead of explaining "Filing Doctrine," the agent file points to PoT.md #1.1.
  - Instead of explaining "How to Verify," the agent file points to S-LEARN-01.

This makes the Agent file a **Connection Hub** rather than a textbook.

5. Proposal: The "R-AGENT" Refactor

We propose a complete refactor of the docs/library/agents/ directory and its supporting ops.

5.1 New Artifact: ops/lib/scripts/agent.sh

We need a dedicated binary for Agent lifecycle management, mirroring skill.sh.

Logic Specification:

1. **harvest Command:**
   - **Input:** --name, --dp, --context (What did this agent do?).
   - **Action:** Creates storage/handoff/agent-draft-XXX.md.
   - **Provenance:** Injects the standard Provenance Block (Timestamp, Git Hash, Friction Context).
2. **promote Command:**
   - **Validation:** Checks for "Context Hazards" (e.g., does the agent file try to load illegal files?).
   - **Validation:** Checks for **PoT Duplication**. (e.g., "Error: Agent draft contains 'No contractions'. This is a global rule. Remove text and use a pointer to PoT.").
   - **Registration:** Assigns R-AGENT-XX, moves to docs/library/agents/, updates INDEX.md.

5.2 Schema: The Pointer-First Agent

The new agent file structure must be rigorously defined.

Section	Content Strategy	Pointer Target (SSOT)
Header	Name, Description, Model	N/A
Provenance	Immutable capture data	ops/lib/scripts/agent.sh
Constitution	"Follow these laws."	PoT.md
Jurisdiction	"Where you can work."	docs/GOVERNANCE.md
Tools	"What you can run."	ops/bin/, tools/lint/
Skills	"What you know."	docs/library/skills/S-LEARN-XX
Tasks	"What you do."	docs/library/tasks/B-TASK-XX

Benefit: This creates an agent that is extremely lightweight in terms of token count (low context cost) but extremely high in "compliance density" because it links strictly to the Canonical Truth.

6. Seclusion: The Clean Room Protocol

The user query anticipates "Seclusion." This is the operational state where "Context Hazards" are mathematically impossible because the hazard files are not even reachable.

6.1 The Seclusion Architecture

In a standard session, ops/bin/llms loads a "Small" or "Full" bundle. In Seclusion, we introduce a **"Project Bundle"**.

- **Mechanism:** ops/bin/project work <id> [1] currently loads OPEN + Platform Dump + Project STELA.
- **Upgrade:** It should load OPEN + Secluded Dump.
  - **Secluded Dump:** Excludes *all* of docs/library/ *except* the specific Agent and Skills required.
  - **Result:** The AI sees PoT.md, TASK.md, projects/my-app/, and R-AGENT-04. It does *not* see R-AGENT-01 or S-LEARN-08.

6.2 Seclusion as a drift-killer

Drift often happens when an AI confuses the rules of one project with another, or applies a "Platform Maintenance" rule (e.g., "Update SoP") to a "Payload" task (e.g., "Fix CSS"). Seclusion eliminates this by physically removing the irrelevant rules from the context window.

7. Implementation Roadmap

Phase 1: The Audit (Current DP)

1. **Dump Analysis:** Confirm docs/library/agents/ contents.
2. **Lint Check:** Run tools/lint/library.sh to map current registrations.

Phase 2: The Tooling (Next DP)

1. **Create ops/lib/scripts/agent.sh:** Clone skill.sh logic. Adapt the provenance block. Add the "PoT Duplication" linter to the promote function.
2. **Update ops/bin/llms:** Add support for dynamic agent injection (e.g., --agent=R-AGENT-01).

Phase 3: The Refactor (The Harvest)

1. **Harvest Current Agents:** "Re-harvest" the existing static agents (01 through 06) into the new format. This requires an Operator to manually strip the PoT duplication and insert the Pointers.
2. **Promote & Replace:** Overwrite the old files with the new Provenance-backed files.

Phase 4: Seclusion (Future DP)

1. **Update ops/bin/project:** Modify the context generation to use "Exclusive Mode" (Seclusion), loading only the allow-listed pointers defined in the Agent file.

8. Closing: The Policy of Truth in Action

This report demonstrates that the "Stela" system is designed to be self-correcting. The "Context Hazard" rules and "Filing Doctrine" are not just bureaucratic hurdles; they are the immune system of the repository. By extending the **Harvest and Promote** pattern—proven in the Skills System—to the Agent System, Stela closes a critical governance gap. Agents will cease to be static, potentially drifting definitions and will become **provenance-backed operational artifacts**.

The resulting **Pointer-First Agents** will be:

- **Lighter:** Fewer tokens, less duplication.
- **Safer:** No context hazards.
- **Proven:** Backed by cryptographic provenance (commit hashes).
- **Secludable:** Ready for the next generation of isolated, high-security project work.

This is the operational realization of the mandate: **"There is only the Policy of Truth."**

Data Appendix: Verified Canon Pointers

- **PoT.md:** [1] (Source of Truth)
- **TASK.md:** [1] (Dispatch Contract)
- **ops/bin/dump:** [1] (State Capture)
- **ops/lib/scripts/skill.sh:** [1] (Provenance Mechanism)
- **docs/library/agents/R-AGENT-01:** [1] (Target for Refactor)
- **docs/library/SKILLS.md:** [1] (Registry Template)
- **Context Hazard:** [1] (Axiom 1.2)

Detailed Analysis: Stela Operational Specs & Agent Protocol

1. Introduction: The Stela "State Machine"

Stela is not merely a codebase; it is a text-based state machine where the file system constitutes the memory, logic, and law of the operation. The **Policy of Truth (PoT)** serves as the immutable kernel, defining the physics of this universe.[1] In this environment, "Agents" are not vague AI personalities; they are distinct, file-backed operational states that must be managed with the same rigor as compiled binary code.

The user's request focuses on a critical evolution of this system: upgrading the **Agent System** (R-AGENT-XX) to match the maturity of the **Skills System** (S-LEARN-XX). This involves adopting "Pointer-first" architecture, strict "Context Hazard" guardrails, and preparing for "Seclusion."

This report dissects the existing machinery of Stela—specifically the ops/ kernel and docs/library/ registries—to engineer a compliant, robust Agent Provenance Protocol.

2. The Ops Kernel: Machinery of Truth

To propose a valid update to the Agent System, we must first understand the machinery that will enforce it. The ops/ directory is the "Run" jurisdiction.[1]

2.1 The "Open/Dump" Loop

The fundamental heartbeat of Stela is the **Open/Dump Loop**.

- **Open (ops/bin/open):** This tool initializes a session. It captures the "Freshness Gate" (Git Commit Hash, Branch Name) and injects the "Intent" and "Dispatch Packet (DP)" into the context.[1] It establishes the *temporal reality* of the agent.
- **Dump (ops/bin/dump):** This tool serializes the repository state into a single text stream for the AI. Crucially, it supports **Scopes** (platform, full, project). This scoping capability is the technical foundation for **Seclusion**—allowing an agent to see *only* what is relevant.[1]

2.2 The Context Bundler (ops/bin/llms)

The ops/bin/llms binary is the gatekeeper of what the AI "knows".[1]

- **Manifest Supremacy:** It reads ops/lib/manifests/CONTEXT.md to decide what files to include.
- **Hazard Exclusion:** It explicitly rejects docs/library/agents, docs/library/tasks, and docs/library/skills. This is the code-level enforcement of the **Context Hazard** axiom.
- **Profiles:** It supports profiles like --profile=architect or --profile=security. This is a primitive form of Seclusion, injecting specific knowledge layers on top of the base context.[1]

Implication for Agents: The new Agent System must integrate with ops/bin/llms. Instead of "Profiles" being hard-coded lists, they should likely be driven by the R-AGENT definition itself.

3. The Skills System: The Gold Standard

The **Skills System** is the benchmark for the proposed Agent update. It has successfully solved the problem of "Hallucinated Capabilities" by enforcing **Provenance**.

3.1 Anatomy of a Skill (S-LEARN)

Reviewing the S-LEARN files (e.g., S-LEARN-01, S-LEARN-07) reveals a rigid schema [1]:

1. **Provenance Block:** Captured: YYYY-MM-DD, Origin: DP-OPS-XXXX, Friction Context: Hot Zone. This proves the skill is real.
2. **Scope:** "Production payload work only."
3. **Invocation Guidance:** "Use this skill when..."
4. **Drift Preventers:** "Stop conditions," "Anti-hallucination."
5. **Procedure:** The executable steps.

3.2 The Harvest/Promote Workflow

The workflow is circular and self-reinforcing [1]:

1. **Work:** A Contractor executes a DP.
2. **Harvest:** The Operator runs ops/lib/scripts/skill.sh harvest. The script scrapes the session/git-state and creates a draft.
3. **Refine:** The draft is reviewed in storage/handoff/.

4. **Promote:** The Operator runs promote. The skill becomes Canon (docs/library/skills/S-LEARN-XX.md).
5. **Register:** The index (docs/library/INDEX.md) is updated.

Insight: This workflow turns "work" into "asset." The user wants to apply this to **Agents**. An Agent shouldn't just be "written"; it should be "harvested" from a session where an AI successfully embodied a specific persona (e.g., "The ruthless code reviewer").

4. Agent System Analysis: The "Static" Flaw

Currently, Agents (docs/library/agents/) are static.

- **Drift Risk:** R-AGENT-01 (Architect) [1] defines how an architect *should* behave. But if the actual LLM models evolve, or if the repository complexity grows, this static file might become obsolete.
- **Redundancy:** Agents currently contain instructions like "No Direct Pushes" or "Linguistic Precision." These are global rules defined in PoT.md. Including them in R-AGENT files violates **Pointer-First** design (Don't Repeat Yourself). If PoT.md changes, every Agent file is now out of date.

4.1 The Pointer-First Imperative

The user explicitly asks for a "Pointer-first agent constitution."

- **Current State:** Content-heavy agent files.
- **Target State:** Context-light, Pointer-heavy agent files.
  - Instead of explaining "Filing Doctrine," the agent file points to PoT.md #1.1.
  - Instead of explaining "How to Verify," the agent file points to S-LEARN-01.

This makes the Agent file a **Connection Hub** rather than a textbook.

5. Proposal: The "R-AGENT" Refactor

We propose a complete refactor of the docs/library/agents/ directory and its supporting ops.

5.1 New Artifact: ops/lib/scripts/agent.sh

We need a dedicated binary for Agent lifecycle management, mirroring skill.sh.

Logic Specification:

1. **harvest Command:**
   - **Input:** --name, --dp, --context (What did this agent do?).
   - **Action:** Creates storage/handoff/agent-draft-XXX.md.
   - **Provenance:** Injects the standard Provenance Block (Timestamp, Git Hash, Friction Context).

2. **promote Command:**
    - **Validation:** Checks for "Context Hazards" (e.g., does the agent file try to load illegal files?).
    - **Validation:** Checks for **PoT Duplication**. (e.g., "Error: Agent draft contains 'No contractions'. This is a global rule. Remove text and use a pointer to PoT.").
    - **Registration:** Assigns R-AGENT-XX, moves to docs/library/agents/, updates INDEX.md.

5.2 Schema: The Pointer-First Agent

The new agent file structure must be rigorously defined.

Section	Content Strategy	Pointer Target (SSOT)
Header	Name, Description, Model	N/A
Provenance	Immutable capture data	ops/lib/scripts/agent.sh
Constitution	"Follow these laws."	PoT.md
Jurisdiction	"Where you can work."	docs/GOVERNANCE.md
Tools	"What you can run."	ops/bin/, tools/lint/
Skills	"What you know."	docs/library/skills/S-LEARN-XX
Tasks	"What you do."	docs/library/tasks/B-TASK-XX

Benefit: This creates an agent that is extremely lightweight in terms of token count (low context cost) but extremely high in "compliance density" because it links strictly to the Canonical Truth.

6. Seclusion: The Clean Room Protocol

The user query anticipates "Seclusion." This is the operational state where "Context Hazards" are mathematically impossible because the hazard files are not even reachable.

6.1 The Seclusion Architecture

In a standard session, ops/bin/llms loads a "Small" or "Full" bundle. In Seclusion, we introduce a **"Project Bundle"**.

- **Mechanism:** ops/bin/project work <id> [1] currently loads OPEN + Platform Dump + Project STELA.
- **Upgrade:** It should load OPEN + Secluded Dump.
  - **Secluded Dump:** Excludes *all* of docs/library/ *except* the specific Agent and Skills required.
  - **Result:** The AI sees PoT.md, TASK.md, projects/my-app/, and R-AGENT-04. It does *not* see R-AGENT-01 or S-LEARN-08.

6.2 Seclusion as a drift-killer

Drift often happens when an AI confuses the rules of one project with another, or applies a "Platform Maintenance" rule (e.g., "Update SoP") to a "Payload" task (e.g., "Fix CSS"). Seclusion eliminates this by physically removing the irrelevant rules from the context window.

7. Implementation Roadmap

Phase 1: The Audit (Current DP)

1. **Dump Analysis:** Confirm docs/library/agents/ contents.
2. **Lint Check:** Run tools/lint/library.sh to map current registrations.

Phase 2: The Tooling (Next DP)

1. **Create ops/lib/scripts/agent.sh:** Clone skill.sh logic. Adapt the provenance block. Add the "PoT Duplication" linter to the promote function.
2. **Update ops/bin/llms:** Add support for dynamic agent injection (e.g., --agent=R-AGENT-01).

Phase 3: The Refactor (The Harvest)

1. **Harvest Current Agents:** "Re-harvest" the existing static agents (01 through 06) into the new format. This requires an Operator to manually strip the PoT duplication and insert the Pointers.
2. **Promote & Replace:** Overwrite the old files with the new Provenance-backed files.

Phase 4: Seclusion (Future DP)

1. **Update ops/bin/project:** Modify the context generation to use "Exclusive Mode" (Seclusion), loading only the allow-listed pointers defined in the Agent file.

8. Closing: The Policy of Truth in Action

This report demonstrates that the "Stela" system is designed to be self-correcting. The

"Context Hazard" rules and "Filing Doctrine" are not just bureaucratic hurdles; they are the immune system of the repository. By extending the **Harvest and Promote** pattern—proven in the Skills System—to the Agent System, Stela closes a critical governance gap. Agents will cease to be static, potentially drifting definitions and will become **provenance-backed operational artifacts**.

The resulting **Pointer-First Agents** will be:

- **Lighter:** Fewer tokens, less duplication.
- **Safer:** No context hazards.
- **Proven:** Backed by cryptographic provenance (commit hashes).
- **Secludable:** Ready for the next generation of isolated, high-security project work.

This is the operational realization of the mandate: **"There is only the Policy of Truth."**

Data Appendix: Verified Canon Pointers

- **PoT.md:** [1] (Source of Truth)
- **TASK.md:** [1] (Dispatch Contract)
- **ops/bin/dump:** [1] (State Capture)
- **ops/lib/scripts/skill.sh:** [1] (Provenance Mechanism)
- **docs/library/agents/R-AGENT-01:** [1] (Target for Refactor)
- **docs/library/SKILLS.md:** [1] (Registry Template)
- **Context Hazard:** [1] (Axiom 1.2)

Works cited

1. dump-full-main-83892f891.txt